



TRABALHO PRÁTICO 2: ARIANAGRAM

Redes de Computadores Junho/2026

Dados do Trabalho

Título do trabalho: Trabalho Prático 2: Arianagram

Alunos: Thales Gregory Vasconcelos Santos – 2018019630

1 Introdução

Este trabalho tem como objetivo apresentar o desenvolvimento do Arianagram, uma rede social minimalista baseada no modelo cliente-servidor, utilizando comunicação via sockets TCP e suporte a múltiplas conexões simultâneas. A proposta central do sistema é permitir que diferentes usuários se conectem a um servidor, publiquem mensagens, sigam outros usuários e recebam notificações em tempo real quando os perfis acompanhados realizarem novas postagens.

A implementação foi desenvolvida em linguagem C, utilizando a interface POSIX para comunicação em rede e a biblioteca 'pthread' para gerenciamento de múltiplas threads. O servidor segue a arquitetura Single-Process, Multi-Thread, na qual um processo principal permanece aguardando novas conexões e, a cada cliente conectado, cria uma thread dedicada para tratar suas mensagens durante toda a sessão.

Além da comunicação tradicional entre cliente e servidor, o projeto também explora o paradigma Publish-Subscribe. Nesse modelo, os usuários podem se inscrever para acompanhar postagens de outros perfis, enquanto o servidor atua como intermediador responsável por armazenar as publicações, controlar a relação entre usuários seguidos e seguidores, e encaminhar notificações push para os clientes conectados.

Ao longo do desenvolvimento, foram aplicados conceitos importantes de redes de computadores, como criação e configuração de sockets TCP, uso de IPv4 e IPv6, envio e recebimento de mensagens binárias de tamanho fixo, tratamento de conexões encerradas, sincronização de estruturas compartilhadas e comunicação assíncrona entre servidor e clientes.

2 Arquitetura do Sistema

2.1 Visão Geral

O sistema desenvolvido segue o modelo cliente-servidor, utilizando sockets TCP para permitir a comunicação entre os usuários e o servidor central. O servidor concentra a lógica principal da aplicação, enquanto os clientes funcionam como interface de interação, enviando comandos e exibindo mensagens recebidas.

A arquitetura do servidor segue o modelo Single-Process, Multi-Thread. O processo principal permanece aguardando novas conexões e, a cada cliente conectado, cria uma thread dedicada para atender aquela sessão. Isso permite que múltiplos usuários interajam simultaneamente com o sistema.

Além do fluxo tradicional de requisição e resposta, o Arianagram também implementa notificações push. Assim, quando um usuário publica uma mensagem, o servidor verifica seus seguidores ativos e envia a postagem diretamente para os clientes conectados que seguem aquele perfil.

2.2 Estrutura dos componentes

2.2.1 Servidor

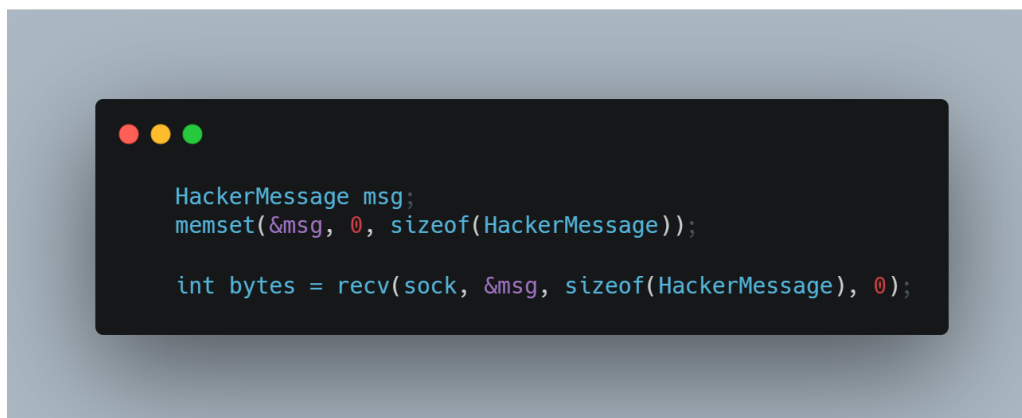
O servidor é responsável por criar e configurar o socket TCP, associá-lo a uma porta com `bind()`, colocá-lo em modo de escuta com `listen()` e aceitar conexões com `accept()`. Para cada novo cliente, é criada uma thread com `pthread_create()`.

Durante a execução, o servidor mantém três estruturas principais: o feed global de mensagens, a tabela de seguidores e a lista de clientes ativos. Essas estruturas permitem armazenar posts, registrar relações de FOLLOW e localizar os sockets dos usuários conectados para envio de `MSG_PUSH`.

Como essas informações são compartilhadas entre diferentes threads, o acesso a elas é protegido com mutexes, evitando inconsistências causadas por acessos concorrentes.

2.2.2 Cliente

Já para o cliente, a implementação se torna bem mais simplificada, utilizando apenas o método `connect()` para estabelecer a conexão com o servidor, e os métodos `send()` e `recv()` para enviar as tentativas de senha e receber os feedbacks do servidor, respectivamente, se aproveitando da struct `HackerMessage` para padronizar o formato das mensagens trocadas entre o cliente e o servidor e garantir a integridade dos dados transmitidos.



```
HackerMessage msg;
memset(&msg, 0, sizeof(HackerMessage));

int bytes = recv(sock, &msg, sizeof(HackerMessage), 0);
```

Figura 1: Definição da variável de armazenamento da mensagem de requisição do cliente.

3 Desafios e Dificuldades

Durante o desenvolvimento do sistema, foram enfrentados alguns desafios e dificuldades, principalmente relacionados à implementação da comunicação via sockets TCP e ao tratamento de erros. Num primeiro momento, o entendimento do modelo cliente-servidor e a configuração correta dos sockets se mostrou um desafio, exigindo uma compreensão aprofundada dos conceitos de redes de computadores e do funcionamento dos protocolos de comunicação.

A pouca prática prévia relacionada à programação em C e à utilização de sockets também con-

tribuiu para a dificuldade inicial, demandando um esforço adicional para aprender e aplicar esses conceitos de forma eficaz.

No entanto, após superados os desafios, a fase de padronização das mensagens utilizando o protocolo previamente definido, também gerou uma certa dificuldade inicial, que foi superada com uma maior rapidez do que a fase de implementação da comunicação via sockets TCP, devido à experiência adquirida durante o desenvolvimento do projeto. Entretanto, mesmo com uma certa dificuldade inicial, foi interessante entender a importância de padronizar as mensagens, fazendo com que o restante do desenvolvimento se mostrasse mais tranquilo e fluído.

Por fim, garantimos a robustez do sistema por meio de um tratamento de erros adequado, utilizando mensagens de erros claras e informativas para o usuário, além de implementar mecanismos de validação para garantir a integridade dos dados transmitidos e recebidos.

4 Conclusão

Assim, concluímos que esse projeto permitiu a aplicação prática de conceitos de redes de computadores, programação em C e desenvolvimento de sistemas utilizando o modelo cliente-servidor. A implementação do sistema *Codebreaker* proporcionou um estudo claro e conciso sobre a comunicação via sockets TCP.